TECHNICAL RAPPORT

train_cnn_v2.py

Industrial Rim Inspection — EfficientNet-B0 + ResNet-50 Ensemble

0. What Is This Project?

This code trains a deep learning model to automatically inspect industrial wheel rims using camera images. The model's job is binary classification: given an image of a rim, decide whether it is GOOD (label 0) or FAULTY (label 1). The target performance is F1 ≥ 0.97 and AUC ≥ 0.97 — very high thresholds suitable for production quality control.

Goal	Detect defective rims automatically from photos — replacing or supporting manual visual inspection on an industrial line.
Input	A folder of rim images + a CSV file (train.csv) linking each image filename to its label (0=good, 1=faulty).
Output	A saved model file (model.pt) that can be loaded by compare.py or predict.py to classify new rim images.

1. Big Picture — How the Code Is Organised

The script is split into clear stages that execute in order:

Stage	Function / Class	What it does
1. Image Preprocessing	extract_rim_crop()	Finds the circular rim in the image using Hough Circles and crops tightly around it
2. Data Augmentation	_make_train_transform()	Applies random flips, rotations, colour jitter, RandAugment, RandomErasing to multiply training diversity
3. Dataset & DataLoader	RimDataset, DataLoader	Feeds images batch-by-batch to the GPU in an efficient pipeline
4. Model Definition	EnsembleModel, SEBlock	Two pre-trained CNNs merged together with an attention layer and a classifier head
5. Loss Function	FocalLossWithLogits	Penalises the model more for hard-to-classify examples
6. Augmentation Mix	mixup_batch(), cutmix_batch()	Blends pairs of images during training to reduce overfitting

Stage	Function / Class	What it does
7. Training Loop	train()	Runs epochs, updates weights, tracks best model, applies EMA and SWA
8. Validation & Metrics	_validate(), _sweep_threshold()	Computes F1, AUC, precision, recall; finds best classification threshold

2. Core Concepts Explained

2.1 Transfer Learning & Pre-trained Models

Training a CNN from scratch requires millions of images and days of GPU time. Transfer learning avoids this: a model already trained on ImageNet (1.2 million general images) already knows how to detect edges, textures, shapes and objects. You only need to teach it the last step: "is this specific rim faulty?"

This code uses TWO pre-trained models:

- EfficientNet-B0 — a compact but powerful architecture designed to be highly efficient. It extracts 1280 feature values from each image.
- ResNet-50 — a deeper, more classical architecture (50 layers). It extracts 2048 feature values per image.

Both backbones have their original classifier removed (replaced by nn.Identity()) so only the raw feature vectors come out. These are concatenated into a single vector of 3328 values that describes each image.

```
self.b0.classifier = nn.Identity()    # remove EfficientNet head
self.resnet.fc     = nn.Identity()    # remove ResNet head
```

2.2 Ensemble Models

An ensemble combines multiple models to produce a better final prediction than any single model alone — just like asking several experts and averaging their opinions. Here, EfficientNet-B0 and ResNet-50 have different architectures and therefore make different types of errors. By merging their features before the final decision, the model is more robust.

```
feat = torch.cat([f_b0, f_rn], dim=1)    # [B, 1280+2048] = [B, 3328]
```

2.3 Squeeze-and-Excitation (SE) Channel Attention [CV-4]

After concatenating the 3328 features, not all of them are equally useful. The SE block learns to re-weight (scale) each feature channel: important channels get amplified, unimportant ones get suppressed. This is a form of attention — the network learns what to focus on.

How SEBlock works step by step:

- Takes in a feature vector of shape (Batch, 3328)
- Compresses it to (Batch, 206) with a linear layer + ReLU — a 'squeeze'

- Expands it back to (Batch, 3328) with another linear + Sigmoid — an 'excitation'
- Multiplies element-wise with the original features — re-weighting

2.4 The Classifier Head

After the SE block, a small neural network makes the final binary decision. Each layer reduces the dimensionality and the model learns which combinations of features indicate a faulty rim:

```
Linear(3328 → 512) → BatchNorm → SiLU → Dropout(0.45)
Linear(512 → 128)  → BatchNorm → SiLU → Dropout(0.30)
Linear(128 → 1)    → single logit (raw score)
```

The output is a single number (logit). Passing it through sigmoid gives a probability from 0 to 1. If probability ≥ threshold → Faulty; otherwise → Good.

Key regularisation elements:

- BatchNorm1d — normalises activations to keep training stable
- SiLU (Swish) — a smooth activation function that outperforms ReLU on many tasks
- Dropout — randomly zeros neurons during training to prevent memorisation

3. Data Pipeline

3.1 Hough Circle Detection for Rim Cropping

Before any deep learning, the code locates the rim in the raw image. Rim images typically show a wheel in a camera frame with background noise. The function extract_rim_crop() does the following:

- Converts the image to greyscale
- Applies Gaussian blur to reduce noise
- Runs HoughCircles — a classical computer vision algorithm that detects circular shapes by voting in parameter space
- If a circle is found: crops a square region around it, adding a small margin that includes the rim edge (the most likely defect location)
- If no circle is found: falls back to the full image
- Resizes the crop to 224×224 pixels

3.2 Data Splitting [ML-5]

The dataset is split into three parts. Critically, the split is stratified — both classes (good/faulty) are represented proportionally in every split. This prevents accidentally putting all faulty rims into training only.

Split	Fraction	Purpose
Train	~90%	Used to update model weights during training
Validation	~10%	Used to measure performance during training — never trained on

Split	Fraction	Purpose
Test	0% (optional)	Final unbiased evaluation — completely held out

The validation split is saved to assets/val_split.csv so that compare.py can use the exact same images — preventing data leakage between experiments.

3.3 Handling Class Imbalance [ML-1]

In quality control datasets, defective items are typically rare — for example 90% good rims vs 10% faulty. If you train on this directly, the model learns to always predict 'good' and achieves 90% accuracy while being useless for detecting faults. Two complementary strategies are used:

Oversampling (_oversample_minority)

Copies of the minority class (faulty rims) are duplicated randomly (with replacement) until both classes have the same count. The model then sees equal numbers of both classes per epoch.

```
extra = [rng.choice(minority) for _ in range(n_extra)]
```

WeightedRandomSampler

Even after oversampling, the DataLoader is given explicit per-sample weights so that each class is sampled equally in every batch. This is a second safety net.

4. Data Augmentation Techniques

Augmentation artificially increases the training set size by creating modified versions of existing images. The model never sees the exact same image twice, so it cannot memorise — it must learn general features.

4.1 Standard Spatial & Colour Augmentations [CV-1]

Augmentation	What it does
RandomHorizontalFlip / VerticalFlip	Mirrors the image — rim defects can appear anywhere
RandomRotation(180)	Rotates up to 180° — rims are circular so any rotation is valid
RandomPerspective	Simulates a camera at a slightly different angle
RandomAffine (shear)	Elastic-like distortion — shears the image to simulate lens distortion
GaussianBlur	Adds slight blur — simulates out-of-focus camera
ColorJitter	Randomly changes brightness, contrast, saturation, hue
RandAugment [CV-5]	Automatically selects and applies 2 random augmentation operations
RandomErasing [CV-1]	Randomly blacks out a rectangle — forces model to use whole image

4.2 Mixup [ML-2]

Mixup creates entirely new virtual images by linearly blending two real images and their labels. If lambda = 0.7, the new image is 70% image A + 30% image B, and the label is also 0.7·labelA + 0.5·labelB. This softens decision boundaries and makes the model less overconfident.

```
mixed_image = λ·img_A + (1-λ)·img_B
mixed_label = λ·label_A + (1-λ)·label_B
```

4.3 CutMix [ML-2]

CutMix is a stronger spatial variant: instead of blending whole images, a rectangular patch is cut from image B and pasted into image A. The label is adjusted proportionally by how much area was replaced. This forces the model to use information from the entire image rather than fixating on one region.

```
mixed[:, :, y1:y2, x1:x2] = imgs_B[:, :, y1:y2, x1:x2]
```

Lambda (λ) is sampled from a Beta(α, α) distribution with α=0.3. Low α values keep λ close to 0 or 1 (mild mixing), while α=1 would give uniform mixing.

4.4 Test-Time Augmentation (TTA) [CV-2]

TTA applies augmentations at inference time. Instead of one prediction per image, 5 predictions are made (original + horizontal flip + vertical flip + 90° rotation + 270° rotation) and their probabilities are averaged. This acts like a mini ensemble and consistently improves accuracy.

```
probs = sigmoid( (logits + logits_hflip + logits_vflip + logits_rot90 +
logits_rot270) / 5 )
```

5. Loss Functions

5.1 Focal Loss [CV-3]

Standard Binary Cross-Entropy (BCE) loss treats all examples equally. But in imbalanced datasets, most examples are easy (correctly classified with high confidence) — they dominate the loss and contribute little useful gradient signal. Focal Loss fixes this by down-weighting easy examples using a modulating factor:

```
Focal Loss = (1 - p_t)^γ × BCE
```

Where p_t is the model's confidence for the correct class and γ (gamma) is a hyperparameter (default = 2.0). When the model is very confident (p_t ≈ 1), the factor $(1-p_t)^2 \approx 0$ so the loss is nearly zero. When the model is wrong (p_t ≈ 0), the factor ≈ 1 and full BCE applies.

5.2 Label Smoothing

Instead of hard labels (0 or 1), the targets are slightly softened: 0 becomes 0.025 and 1 becomes 0.975. This prevents the model from becoming overconfident and improves calibration. The formula used:

```
target_smooth = target × (1 - ε) + 0.5 × ε      [ε = 0.05]
```

5.3 Positive Class Weighting

The loss function also receives a pos_weight tensor computed from the class frequencies. If there are 9× more good rims than faulty ones, faulty rim errors are penalised 9× more heavily, giving the optimizer a strong signal to not ignore them.

```
pos_weight = n_good / n_faulty
```

6. Optimisation & Learning Rate Scheduling

6.1 Two-Phase Training Strategy

Training is split into two phases to make fine-tuning stable:

Phase	Description
Phase 1: Warmup (epochs 1–12)	Backbone weights are frozen. Only the SE block and classifier head are trained. Learning rate is high (lr×10). Scheduler: OneCycleLR — ramps up then down in one cycle.
Phase 2: Fine-tune (epochs 13–60)	Backbone weights are unfrozen. Differential learning rates: backbone gets lr×0.05 (very small to not destroy pre-trained features), head gets full lr. Scheduler: CosineAnnealingWarmRestarts.

6.2 AdamW Optimizer

AdamW (Adam with decoupled Weight Decay) is the standard optimizer for fine-tuning transformers and CNNs. It adapts the learning rate per-parameter using estimates of gradient mean and variance, plus adds L2 weight regularisation to prevent overfitting. Weight decay = 1e-4.

6.3 Cosine Annealing with Warm Restarts (SGDR) [ML-3]

After Phase 1, the learning rate follows a cosine curve: it decays smoothly from lr to lr×0.001, then periodically 'restarts' back to a higher value. Each restart allows the optimizer to escape local minima and explore different regions of the loss landscape. T_0 is set to one third of the remaining training epochs so there are roughly 3 restart cycles.

6.4 Gradient Accumulation

With a batch size of 16 and accum_steps=4, gradients are accumulated over 4 batches before an optimizer step. Effectively this simulates a batch size of 64 without needing 4× the GPU memory — important on limited hardware.

6.5 Mixed Precision Training (AMP)

PyTorch's autocast runs forward passes in float16 (half precision) on CUDA GPUs. This halves memory usage and speeds up training on modern GPUs (RTX, A100, etc.) with negligible accuracy loss. GradScaler compensates for the reduced numerical range during backpropagation.

7. Regularisation Techniques

Regularisation prevents overfitting — where the model memorises training data but fails on new images.

7.1 Exponential Moving Average (EMA)

EMA maintains a 'shadow' copy of the model whose weights are a weighted average of all past weights. After each batch:

```
shadow_weight = decay × shadow_weight + (1 - decay) × current_weight   [decay = 0.9995]
```

The shadow model changes very slowly — it is much more stable than the model being actively trained. During validation, the shadow weights are temporarily loaded. EMA models almost always outperform the raw trained model, especially late in training.

7.2 Stochastic Weight Averaging (SWA) [ML-4]

SWA averages the model weights from multiple points along the training trajectory — from epoch 40 onwards. While EMA gives a smooth exponential average, SWA gives a simple arithmetic average of periodic checkpoints. The averaged solution tends to lie in a wider, flatter minimum of the loss landscape that generalises better.

After SWA, BatchNorm statistics are recalibrated with 50 forward passes over training data because the averaged weights are never seen in a single training run.

7.3 Early Stopping

If the validation F1 does not improve for 12 consecutive epochs, training stops. This prevents wasting compute and avoids the overfitting that happens in the late stages of unnecessary training.

8. Evaluation Metrics

Accuracy alone is misleading for imbalanced datasets — a model that always predicts 'Good' gets high accuracy but zero usefulness. These metrics are used instead:

Metric	Formula	What it measures
Precision	TP / (TP + FP)	Of all predicted faulty rims, how many were actually faulty?
Recall (Sensitivity)	TP / (TP + FN)	Of all actual faulty rims, how many did we catch?

Metric	Formula	What it measures
F1 Score	2·TP / (2·TP + FP + FN)	Harmonic mean of Precision & Recall — best single metric for imbalanced tasks
AUC-ROC	Area under ROC curve	Measures separability across all thresholds — 1.0 = perfect, 0.5 = random

8.1 Threshold Sweeping [ML-6]

The model outputs a probability. The default decision threshold (0.5) is almost never optimal for imbalanced tasks. _sweep_threshold() and _quick_sweep() test every threshold from 0.05 to 0.90 in steps of 0.05 and report the one that maximises F1. This threshold is saved in the checkpoint and printed at the end of training.

In industrial inspection it is often better to have lower precision (some false alarms) and higher recall (never miss a defect). The threshold allows you to tune this trade-off after training.

9. Understanding the Confusion Matrix

Every validation step computes TP, TN, FP, FN — the four cells of the confusion matrix:

	Predicted: Good	Predicted: Faulty
Actual: Good	TN — Correct rejection	FP — False alarm (Type I error)
Actual: Faulty	FN — Missed defect (Type II error) ⚠	TP — Correct detection

In industrial inspection, FN (missed defects) is the most dangerous error — a faulty rim reaches the customer. FP (false alarms) waste time but are safe. Lowering the threshold increases Recall (fewer FN) at the cost of Precision (more FP).

10. Additional Technical Features

10.1 Gradient Clipping

nn.utils.clip_grad_norm_(model.parameters(), 1.0) caps the maximum gradient norm at 1.0 before each optimizer step. This prevents 'gradient explosions' where a very large gradient update destroys the learned weights in one step — especially important in the early epochs.

10.2 BatchNorm Recalibration

When the EMA shadow model is saved as a checkpoint, its BatchNorm running statistics (mean and variance) are recalibrated by passing 50 training batches through it in eval mode. This is

necessary because the EMA weights were never directly trained — their BatchNorm statistics may not reflect actual training data distribution.

10.3 TensorBoard Logging

If tensorboard is installed, training/validation metrics are logged to runs/rim_v3/ after each epoch. Running tensorboard --logdir runs/ lets you watch loss, F1, AUC, precision, recall, and the optimal threshold evolve in real time via a web browser.

10.4 Checkpoint Saving & Resuming

Whenever validation F1 improves, a full checkpoint is saved containing the model weights, optimizer state, epoch number, best F1, and best threshold. Training can be resumed from any checkpoint with --resume path/to/model.pt.

10.5 ImageNet Normalisation

All images are normalised with the ImageNet mean and standard deviation before being fed to the models. This is required because EfficientNet-B0 and ResNet-50 were originally trained on ImageNet-normalised data — using the same normalisation ensures the pre-trained feature detectors work correctly.

```
T.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
```

11. Key Hyperparameters — What They Control

Parameter	Default	Effect
--epochs	60	Total training passes over the dataset
--warmup-epochs	12	Epochs where backbone is frozen
--batch	16	Images per gradient update (×4 accumulation = effective 64)
--lr	1e-4	Base learning rate for head; backbone gets 5% of this
--ema-decay	0.9995	Higher = slower EMA update (more stable shadow model)
--label-smooth	0.05	Label smoothing epsilon (0 = hard labels)
--mixup-alpha	0.3	Controls Beta distribution for Mixup/CutMix (higher = more mixing)
--focal-gamma	2.0	Focal loss exponent (0 = plain BCE, higher = more focus on hard examples)
--early-stop	12	Epochs without improvement before stopping
--swa-start	40	Epoch at which SWA weight averaging begins
--img-size	224	Input image resolution (pixels × pixels)

12. Complete Training Flow — Step by Step

Here is the exact sequence of events when you run: python train_cnn_v2.py

1	**CLI parsing**	argparse reads all hyperparameters from the command line (or uses defaults)
2	**Seed everything**	random, numpy and torch seeds are fixed for reproducibility
3	**Load CSV**	_load_samples() reads train.csv and builds a list of (image_path, label) pairs
4	**Stratified split**	_split_samples() divides into train/val/test, keeping class ratios equal in each
5	**Save val_split.csv**	The exact validation image list is saved for use by compare.py [ML-5]
6	**Oversample**	_oversample_minority() duplicates faulty rim samples until both classes match [ML-1]
7	**Build datasets**	RimDataset objects wrap each split with its respective transforms
8	**Build dataloaders**	WeightedRandomSampler ensures equal class frequency per batch
9	**Build model**	EnsembleModel (EfficientNet-B0 + ResNet-50 + SEBlock + Classifier) loaded with ImageNet weights
10	**Set up EMA**	A shadow model copy initialised for EMA tracking
11	**Set up Focal Loss**	pos_weight computed from class frequencies; FocalLossWithLogits instantiated
12	**Phase 1 (epochs 1–12)**	Backbone frozen. Only head + SE trained with OneCycleLR at high LR
13	**Phase 2 (epochs 13–60)**	Backbone unfrozen. Differential LRs. CosineAnnealingWarmRestarts scheduler
14	**Per-batch: Mixup or CutMix**	Each batch is randomly augmented with either Mixup or CutMix [ML-2]
15	**Per-batch: AMP forward pass**	autocast runs forward in fp16 for speed; loss computed with Focal Loss
16	**Per-batch: accumulation**	Gradients accumulate over 4 steps before optimizer update
17	**EMA update**	Shadow model weights updated after each optimizer step
18	**Validation with TTA**	After each epoch, _validate() + _quick_sweep() run on val set with TTA [CV-2]
19	**SWA update**	From epoch 40, swa_model.update_parameters(model) averages weights [ML-4]
20	**Best checkpoint save**	If F1 improves, checkpoint saved with BN recalibrated on EMA model
21	**Early stopping check**	If no improvement for 12 epochs, training ends early
22	**SWA finalisation**	BN stats recalibrated; SWA model saved; compared to EMA model on val [ML-4]
23	**Final threshold sweep**	Best checkpoint reloaded; full threshold sweep printed; optimal threshold displayed [ML-6]

13. All Techniques — Lecture Mapping

The code explicitly maps each technique to lecture material using [ML-x] and [CV-x] tags:

Tag	Technique	Key Concept
ML-1	SMOTE Oversampling	Imbalanced classes → oversample minority to prevent bias toward majority
ML-2	Mixup + CutMix	Beta-distribution blending of pairs → reduces overconfidence, improves generalisation
ML-3	SGDR (CosineWarmRestarts)	Periodic LR resets help escape local minima in non-convex loss landscapes
ML-4	Stochastic Weight Averaging	Averaging weights across epochs finds flatter minima that generalise better
ML-5	Train/Val/Test separation	Strict split prevents data leakage between model selection and final evaluation
ML-6	Threshold tuning	Always tune decision threshold for the cost structure of your task, never assume 0.5
CV-1	RandomErasing + ElasticTransform	Spatial perturbations create virtual training samples and improve generalisation
CV-2	Test-Time Augmentation	Averaging predictions over multiple views acts as an ensemble at no training cost
CV-3	Focal Loss	Down-weights easy examples so training focuses on hard/rare cases
CV-4	Squeeze-and-Excitation attention	Channel attention in CNNs — lets the network re-weight feature importance
CV-5	RandAugment	Automatically learns a good augmentation policy — data-driven augmentation selection